

Assignment-19.2

2303A51303

Shaik Sabina

Batch:05

Task 1 – Python to Java Conversion

Prompt: Convert the following Python file-handling program into Java.

Requirements:

- Use FileReader, BufferedReader, FileWriter, BufferedWriter
- Handle exceptions properly (try-catch or try-with-resources)
- Keep the same logic and output
- Add comments in Java code

Also provide:

- Sample input file
- Expected output file
- Console output
- Steps to compile and run Java

Python code:

```
[1] 0s ▶ print("# Python code provided by the user")
with open("input.txt", "w") as file:
    file.write("Hello Amulya\n")
    file.write("Welcome to File Handling in Python\n")
    file.write("This is a sample file\n")

with open("input.txt", "r") as infile, open("output.txt", "w") as outfile:
    print("Reading from input.txt...\n")

    for line in infile:
        print(line.strip())
        outfile.write(line.upper())

print("\nData successfully written to output.txt")

# Display content of input.txt and output.txt for verification
print("\n--- Content of input.txt ---")
with open("input.txt", "r") as f:
    print(f.read())

print("\n--- Content of output.txt ---")
with open("output.txt", "r") as f:
    print(f.read())
```

Java code:

```
▼ Main.java (Java Code)

[2]
✓ 0s %writetfile Main.java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        // Step 1: Write data to the input file
        // Using try-with-resources to ensure FileWriter is closed automatically
        try (BufferedWriter writer = new BufferedWriter(new FileWriter("input.txt"))) {
            writer.write("Hello Amulya");
            writer.newLine(); // Add a new line character
            writer.write("Welcome to File Handling in Python");
            writer.newLine();
            writer.write("This is a sample file");
            writer.newLine();
            System.out.println("Input file 'input.txt' created successfully.");
        } catch (IOException e) {
            System.err.println("Error writing to input.txt: " + e.getMessage());
        }

        System.out.println("\nReading from input.txt...\n");

        // Step 2: Read from input file and write to output file (convert to uppercase)
        // Using try-with-resources for both FileReader/BufferedReader and FileWriter/BufferedWriter
        try (
            BufferedReader reader = new BufferedReader(new FileReader("input.txt"));
            BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt"))
        ) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line); // Print to console (equivalent to Python's print(line.strip()))
                writer.write(line.toUpperCase()); // Write uppercase to output file
                writer.newLine(); // Add a new line character
            }
            System.out.println("\nData successfully written to output.txt");
        } catch (IOException e) {
            System.err.println("Error processing files: " + e.getMessage());
        }
    }
}

Writing Main.java

[3]
✓ 2s !javac Main.java

[4]
✓ 0s !java Main

... Input file 'input.txt' created successfully.

Reading from input.txt...
```

Output:

```
Input file 'input.txt' created successfully.

Reading from input.txt...

Hello Amulya
Welcome to File Handling in Python
This is a sample file

Data successfully written to output.txt

▶ print("--- Content of input.txt (after Java run) ---")
  !cat input.txt

print("\n--- Content of output.txt (after Java run) ---")
!cat output.txt

... --- Content of input.txt (after Java run) ---
Hello Amulya
Welcome to File Handling in Python
This is a sample file

--- Content of output.txt (after Java run) ---
HELLO AMULYA
WELCOME TO FILE HANDLING IN PYTHON
THIS IS A SAMPLE FILE
```

Explanation:

- Translate Python file operations (open, read, write) into Java using classes like `BufferedReader` and `BufferedWriter`.
- Maintain the same logic, file content, and console output.
- Handle errors properly using Java exception handling (try-catch or try-with-resources).
- Ensure the Java program works exactly like the Python version when tested with the same input file.

It is about rewriting a Python file-handling program in Java without changing its functionality.

Task 2 – C++ to Python Function Conversion

Prompt: Convert the following C++ sorting program into Python.

Requirements:

- Keep the same sorting logic (Bubble Sort / Selection Sort, etc.)
- Ensure correct and efficient Python syntax
- Maintain the same input and output behavior
- Add comments explaining the code

Also provide:

- Sample input
- Expected output
- At least 2–3 test cases to verify correctness

C++ code:

```
%%writefile main.cpp
#include <iostream>
using namespace std;
int main() {
    int n;
    // Input size
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    // Input elements
    cout << "Enter elements:\n";
    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    // Bubble Sort logic
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                // Swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    // Output sorted array
    cout << "Sorted array:\n";
    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
}
```

```

    cin >> arr[i];
}
// Bubble Sort logic
for(int i = 0; i < n - 1; i++) {
    for(int j = 0; j < n - i - 1; j++) {
        if(arr[j] > arr[j + 1]) {
            // Swap
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
// Output sorted array
cout << "Sorted array:\n";
for(int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
return 0;
}

```

Python code:

```

# Python program to implement Bubble Sort

def bubble_sort(arr):
    """
    Sorts a list of numbers using the Bubble Sort algorithm.
    """
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place, so we don't need to check them
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Pythonic swap
    return arr

# --- Main part of the program ---

def main():
    # Input size of the array
    while True:
        try:
            n_str = input("Enter number of elements: ")
            n = int(n_str)
            if n < 0:
                print("Number of elements cannot be negative. Please try again.")
            else:
                break

```

```

    except ValueError:
        print("Invalid input. Please enter an integer.")

    if n == 0:
        print("Sorted array: []")
        return

    # Input elements into a list
    arr = []
    print("Enter elements (one by one, then press Enter):")
    for i in range(n):
        while True:
            try:
                element_str = input(f"Element {i + 1}: ")
                element = int(element_str)
                arr.append(element)
                break
            except ValueError:
                print("Invalid input. Please enter an integer.")

    # Sort the array using bubble sort
    sorted_arr = bubble_sort(arr)

    # Output the sorted array
    print("Sorted array:")
    # Convert each element to string and join them with a space
    print(" ".join(map(str, sorted_arr)))

```

Output:

```

# Call the main function
if __name__ == "__main__":
    main()

Enter number of elements: 6
Enter elements (one by one, then press Enter):
Element 1: 12
Element 2: 11
Element 3: 10
Element 4: 13
Element 5: 9
Element 6: 15
Sorted array:
9 10 11 12 13 15

```

Explanation:

- Translate C++ syntax into Python while keeping the same sorting logic.
- Ensure the Python program produces the same sorted output.

- Test the program with different inputs to verify correctness.

It is about rewriting a C++ sorting program in Python without changing how it works.

Task 3 – SQL to Pandas Query

Prompt: Convert the following SQL query (with GROUP BY and aggregate functions) into equivalent Pandas code.

Requirements:

- Use pandas DataFrame operations
- Use groupby() and appropriate aggregation functions (sum, mean, count, etc.)
- Maintain the same logic and output as the SQL query
- Write clean and well-commented Python code

Also provide:

- Sample dataset (as a DataFrame)
- Expected output
- Explanation of how SQL maps to Pandas

SQL query:

```
-- Table: sales
-- Columns: department, employee_name, sales_amount

SELECT
    department,
    COUNT(employee_name) AS total_employees,
    SUM(sales_amount) AS total_sales,
    AVG(sales_amount) AS average_sales
FROM sales
GROUP BY department;
```


Pandas query:

```
import pandas as pd

# 1. Create a sample dataset (equivalent to the 'sales' table)
data = {
    'department': ['Sales', 'Marketing', 'Sales', 'HR', 'Marketing', 'Sales', 'HR', 'Sales'],
    'employee_name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi'],
    'sales_amount': [1000, 500, 1200, 300, 700, 900, 400, 1100]
}
sales_df = pd.DataFrame(data)

print("--- Sample Sales DataFrame ---")
display(sales_df)
```

```
# 2. Convert the SQL query to Pandas code
# SELECT
#     department,
#     COUNT(employee_name) AS total_employees,
#     SUM(sales_amount) AS total_sales,
#     AVG(sales_amount) AS average_sales
# FROM sales
# GROUP BY department;

pandas_result = sales_df.groupby('department').agg(
    total_employees=('employee_name', 'count'), # COUNT(employee_name)
    total_sales=('sales_amount', 'sum'),        # SUM(sales_amount)
    average_sales=('sales_amount', 'mean')      # AVG(sales_amount)
).reset_index() # reset_index() brings 'department' back as a column

print("\n--- Expected Output (Pandas Result) ---")
display(pandas_result)
```

Output:

```
... --- Sample Sales DataFrame ---
```

	department	employee_name	sales_amount
0	Sales	Alice	1000
1	Marketing	Bob	500
2	Sales	Charlie	1200
3	HR	David	300
4	Marketing	Eve	700
5	Sales	Frank	900
6	HR	Grace	400
7	Sales	Heidi	1100

```
--- Expected Output (Pandas Result) ---
```

	department	total_employees	total_sales	average_sales
0	HR	2	700	350.0
1	Marketing	2	1200	600.0
2	Sales	4	4200	1050.0

Explanation:

- Translate SQL operations into Pandas using `groupby()` and aggregation functions like `sum()`, `mean()`, and `count()`.
- Maintain the same logic and results as the SQL query.
- Verify that both SQL and Pandas produce identical outputs.

It is about rewriting a SQL aggregation query using Pandas without changing the result.

Task 4 – Real-Time Application: Multi-Language Snippet

Converter

Prompt: Convert the following code from to for a real-world application.

Requirements:

- Preserve the same logic and functionality
- Use appropriate frameworks/libraries (e.g., Flask → Spring Boot, Python → C++)
- Ensure the code is fully executable
- Write clean, well-structured, and commented code

Also provide:

1. Converted code in target language
2. Sample input and output
3. Steps to run both programs
4. Comparison of both implementations (syntax, libraries, performance, execution behavior)

Source Code:

```
# Python source code provided by the user

def process_data(input_file, output_file):
    try:
        with open(input_file, "r") as infile, open(output_file, "w") as outfile:
            total = 0
            count = 0

            for line in infile:
                try:
                    num = int(line.strip())
                    total += num
                    count += 1
                except ValueError:
                    print(f"Warning: Skipping non-integer line in input file: {line.strip()}")

            average = total / count if count != 0 else 0

            outfile.write(f"Total: {total}\n")
            outfile.write(f"Average: {average}\n")

            print("Processing completed successfully!")

    except FileNotFoundError:
        print("Error: Input file not found")
    except Exception as e:
        print(f"Error: {e}")
```

```

    print(f"Error: {e}")

# Run the program
process_data("input.txt", "output_python.txt") # Using a different output file to avoid conflict

print("\n--- Content of input.txt (for Python) ---")
with open("input.txt", "r") as f:
    print(f.read())

print("\n--- Content of output_python.txt (after Python run) ---")
with open("output_python.txt", "r") as f:
    print(f.read())

```

Java code:

```

%%writefile Main.java
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class Main {

    public static void processData(String inputFileName, String outputFileName) {
        long total = 0; // Use long to prevent potential overflow for sum
        int count = 0;

        try (BufferedReader infile = new BufferedReader(new FileReader(inputFileName));
             BufferedWriter outfile = new BufferedWriter(new FileWriter(outputFileName))) {

            String line;
            while ((line = infile.readLine()) != null) {
                try {
                    int num = Integer.parseInt(line.trim());
                    total += num;
                    count++;
                } catch (NumberFormatException e) {
                    System.err.println("Warning: Skipping non-integer line in input file: " + line);
                }
            }

            double average = (count != 0) ? (double) total / count : 0.0;

```

```

        outfile.write("Total: " + total + "\n");
        outfile.write("Average: " + average + "\n");

        System.out.println("Processing completed successfully!");

    } catch (IOException e) {
        if (e instanceof java.io.FileNotFoundException) {
            System.err.println("Error: Input file not found");
        } else {
            System.err.println("Error during file processing: " + e.getMessage());
        }
    } catch (Exception e) {
        System.err.println("An unexpected error occurred: " + e.getMessage());
    }
}

public static void main(String[] args) {
    processData("input.txt", "output.txt");
}
}

```

Overwriting Main.java

```

▶ with open("input.txt", "w") as file:
    file.write("10\n")
    file.write("20\n")
    file.write("30\n")
    file.write("40\n")
    file.write("50\n")
print("Created input.txt for Java execution.")

# Display content of input.txt
print("\n--- Content of input.txt (for Java) ---")
with open("input.txt", "r") as f:
    print(f.read())

```

Output:

```
Created input.txt for Java execution.
```

```
--- Content of input.txt (for Java) ---
```

```
10
```

```
20
```

```
30
```

```
40
```

```
50
```

```
!javac Main.java
```

```
print("Java code compiled successfully.")
```

```
Java code compiled successfully.
```



```
!java Main
```

```
print("Java code executed.")
```

```
... Processing completed successfully!  
Java code executed.
```

```
print("\n--- Content of output.txt (after Java run) ---")
```

```
!cat output.txt
```

```
--- Content of output.txt (after Java run) ---
```

```
Total: 150
```

```
Average: 30.0
```

Explanation:

- Translate code (e.g., Python to Java or C++) without changing its logic.
- Ensure both versions produce the same output.
- Test both programs to confirm identical behavior.
- Compare differences in syntax, libraries, and execution.

It is about implementing the same application in two different languages and analyzing the differences.

Task -5: (Basic Function Translation (Python to C++))

Prompt: Convert the following Python function into an equivalent C++ program.

Requirements:

- Maintain the same logic (check if number is even or odd)
- Use correct C++ syntax
- Handle user input properly
- Display output clearly (Even/Odd)
- Add comments in the code

Also provide:

- Sample inputs (at least 3 test cases)
- Expected outputs
- Steps to compile and run the C++ program

Python code:

```
# Function to check even or odd
def check_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"

# Taking input from user
number = int(input("Enter a number: "))

# Calling function and displaying result
result = check_even_odd(number)
print("The number is:", result)
```


C++ code:

```
%%writefile even_odd.cpp
#include <iostream>

// Function to check if a number is even or odd
// Takes an integer 'num' as input
// Returns nothing (void) as it prints the result directly
void check_even_odd(int num) {
    // Check if the number is divisible by 2 with no remainder
    if (num % 2 == 0) {
        // If remainder is 0, the number is Even
        std::cout << "Even" << std::endl;
    } else {
        // If remainder is not 0, the number is Odd
        std::cout << "Odd" << std::endl;
    }
}

int main() {
    int number; // Declare an integer variable to store user input

    // Prompt the user to enter a number
    std::cout << "Enter a number: ";
    // Read the integer input from the user
    std::cin >> number;

    // Call the check_even_odd function with the user's number
    std::cout << "The number is: ";
    check_even_odd(number); // Display the result
}
```

```
// Call the check_even_odd function with the user's number
std::cout << "The number is: ";
check_even_odd(number); // Display the result

return 0; // Indicate successful program execution
}
```

Writing even_odd.cpp

```
print("### Compile the C++ code")
print("Now, compile the `even_odd.cpp` file using `g++`. This will create an executable file named `even_odd`.")
!g++ even_odd.cpp -o even_odd
```

```
### Compile the C++ code
Now, compile the `even_odd.cpp` file using `g++`. This will create an executable file named `even_odd`.
```

```
print("### Test Case 1: Even Number")
print("Input: 4")
print("Expected Output: The number is: Even")
!echo "4" | ./even_odd
```


Output:

```
### Test Case 1: Even Number
Input: 4
Expected Output: The number is: Even
Enter a number: The number is: Even
```

```
print("### Test Case 2: Odd Number")
print("Input: 7")
print("Expected Output: The number is: Odd")
!echo "7" | ./even_odd
```

```
### Test Case 2: Odd Number
Input: 7
Expected Output: The number is: Odd
Enter a number: The number is: Odd
```

```
print("### Test Case 3: Zero")
print("Input: 0")
print("Expected Output: The number is: Even")
!echo "0" | ./even_odd
```

```
### Test Case 3: Zero
Input: 0
Expected Output: The number is: Even
Enter a number: The number is: Even
```

Explanation: Translate the Python logic into correct C++ syntax.

- Handle user input and display output properly.
- Ensure both programs give the same result for different test cases.

It is about rewriting a basic Python function in C++ while maintaining the same functionality and output.

